React.js

Day 1 — Introduction to React

Complete Study Notes with Examples

Topics: What is React • JSX • Components • Props • State • Events

2025 Edition

■ Table of Contents

■ 1. What is React?

Definition

React is an open-source **JavaScript library** developed and maintained by **Meta (Facebook)** for building fast, interactive **User Interfaces (UIs)**. Released in 2013, it has become one of the most popular front-end technologies in the world.

Library vs Framework

Aspect	React (Library)	Angular (Framework)
Scope	Handles UI layer only	Full MVC framework
Flexibility	High — choose your own tools	Opinionated, structured
Learning curve	Moderate	Steeper
Data binding	One-way	Two-way

The Virtual DOM

React introduces the concept of a **Virtual DOM** — a lightweight JavaScript copy of the real DOM. When state changes, React:

- Creates a new Virtual DOM tree
- Compares it with the previous version (**diffing**)
- Calculates the minimum number of changes needed (**reconciliation**)
- Updates **only** the changed parts of the real DOM

■ **Tip:** This makes React apps extremely fast compared to direct DOM manipulation.

Key Features of React

- **Component-Based Architecture** — UI is split into small, reusable pieces
- **Declarative** — You describe what the UI should look like, React figures out how
- **Unidirectional Data Flow** — Data flows from parent to child via props
- **JSX** — Lets you write HTML-like syntax inside JavaScript
- **Rich Ecosystem** — React Router, Redux, Next.js, and more

■■ 2. Setting Up React

Prerequisites

- Node.js (v18+) and npm installed — download from nodejs.org
- A code editor — VS Code recommended
- Basic knowledge of HTML, CSS, and JavaScript

Method 1 — Create React App (Official starter)

Create React App (CRA) sets up a full React project with zero configuration.

Terminal

```
npx create-react-app my-app
cd my-app
npm start
```

Method 2 — Vite (Faster, recommended for new projects)

Terminal

```
npm create vite@latest my-app -- --template react
cd my-app
npm install
npm run dev
```

Project File Structure (CRA)

File Structure

```
my-app/
■■■ public/
■   ■■■ index.html        ← Single HTML file (SPA entry point)
■■■ src/
■   ■■■ App.js            ← Root component
■   ■■■ App.css           ← App styles
■   ■■■ index.js          ← ReactDOM renders App here
■   ■■■ components/       ← Your custom components go here
■■■ package.json          ← Project metadata & dependencies
■■■ node_modules/         ← Installed packages (auto-generated)
```

■■ **Note:** Never manually edit node_modules. Run 'npm install' to restore it.

index.js — The Entry Point

src/index.js

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
```

```
);
```

This file mounts the entire React application into the ** ** element inside public/index.html.

■ 3. JSX — JavaScript XML

What is JSX?

JSX is a **syntax extension** for JavaScript that looks like HTML. It is **not** real HTML — Babel compiles it into regular JavaScript function calls before the browser sees it.

JSX vs Plain JavaScript

Comparison

```
// WITH JSX (readable)
const element = <h1 className="title">Hello, World!</h1>;

// WITHOUT JSX (what Babel compiles it to)
const element = React.createElement(
  'h1',
  { className: 'title' },
  'Hello, World!'
);
```

JSX Rules — Important!

- **Return a single root element** — Wrap siblings in a or <></> (Fragment)
- **Close all tags** — Self-closing tags like <img /> and <br /> are required
- **Use className** instead of class (class is a reserved word in JS)
- **Use htmlFor** instead of for in <label> elements
- **camelCase** for all HTML attributes: onclick → onClick, tabindex → tabIndex

JSX Rules Example

```
// ■ Correct JSX
function Welcome() {
  return (
    <>
      <h1 className="title">Welcome</h1>
      <p>This is a paragraph.</p>
      <input type="text" />    {/* Self-closing */}
    </>
  );
}

// ■ Wrong — multiple root elements
function Wrong() {
  return (
    <h1>Title</h1>
    <p>Paragraph</p>    // Error!
  );
}
```

Expressions in JSX

Use **curly braces { }** to embed any valid JavaScript expression inside JSX.

Expressions in JSX

```
const name = "Alice";
const age  = 25;
const isLoggedIn = true;

function Profile() {
  return (
    <div>
      <h2>Hello, {name}!</h2>                    {/* Variable */}
      <p>Age: {age * 1}</p>                       {/* Expression */}
      <p>Status: {isLoggedIn ? "Online" : "Offline"}</p>   {/* Ternary */}
      <p>Current year: {new Date().getFullYear()}</p>      {/* Method */}
    </div>
  );
}
```

Conditional Rendering

Conditional Rendering

```
function Greeting({ isLoggedIn }) {
  // Method 1: if/else outside JSX
  if (!isLoggedIn) return <p>Please log in.</p>;

  // Method 2: Ternary operator (inline)
  return (
    <div>
      {isLoggedIn ? <h1>Welcome back!</h1> : <h1>Hello, Stranger!</h1>}

      {/* Method 3: Short-circuit (&&) — renders only if true */}
      {isLoggedIn && <button>Logout</button>}
    </div>
  );
}
```

■ 4. Components

Components are the **building blocks** of a React application. A component is a JavaScript function (or class) that accepts inputs (**props**) and returns JSX.

Functional Components (Modern — Recommended)

Functional Component

```
// Simple functional component
function Greeting() {
  return <h1>Hello, React!</h1>;
}

// Arrow function syntax (equally valid)
const Greeting = () => {
  return <h1>Hello, React!</h1>;
};

// Shorthand with implicit return
const Greeting = () => <h1>Hello, React!</h1>;
```

Class Components (Legacy — still important to know)

Class Component

```
import React, { Component } from 'react';

class Greeting extends Component {
  render() {
    return <h1>Hello from a Class Component!</h1>;
  }
}

export default Greeting;
```

■ **Tip:** Use functional components for all new code. Class components are still found in older codebases.

Naming Convention

- Component names **must start with a capital letter**
- **MyComponent** — React treats it as a component
- **mycomponent** — React treats it as an HTML tag (and fails)

Exporting & Importing Components

Export / Import

```
// Button.js — Named export
export function Button() {
  return <button>Click Me</button>;
}

// App.js — Named import
import { Button } from './Button';
```

```
// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// Card.js — Default export
function Card() {
  return <div className="card">Card content</div>;
}
export default Card;

// App.js — Default import (any name works)
import Card from './Card';
```

Composing Components

Component Composition

```
function Header() {
  return <header><h1>My App</h1></header>;
}

function Footer() {
  return <footer><p>© 2025</p></footer>;
}

function App() {
  return (
    <div>
      <Header />      {/* Composing components */}
      <main><p>Main content here</p></main>
      <Footer />
    </div>
  );
}

export default App;
```

■ 5. Props (Properties)

Props are the mechanism for passing **data from parent to child** components. They are **read-only** — a component must never modify its own props.

Passing and Receiving Props

Props Example

```
// Parent — passes props as HTML attributes
function App() {
  return (
    <div>
      <UserCard name="Alice" age={25} isAdmin={true} />
      <UserCard name="Bob"   age={30} isAdmin={false} />
    </div>
  );
}

// Child — receives props as an object
function UserCard(props) {
  return (
    <div className="card">
      <h2>{props.name}</h2>
      <p>Age: {props.age}</p>
      {props.isAdmin && <span>■ Admin</span>}
    </div>
  );
}
```

Destructuring Props (Cleaner Syntax)

Destructured Props

```
// Destructure in the parameter list
function UserCard({ name, age, isAdmin }) {
  return (
    <div>
      <h2>{name}</h2>
      <p>Age: {age}</p>
      {isAdmin && <span>Admin</span>}
    </div>
  );
}
```

Default Props

Default Props

```
function Button({ label = "Click Me", color = "blue" }) {
  return (
    <button style={{ backgroundColor: color }}>
      {label}
    </button>
  );
}
```

```
// Usage
<Button />                      // Shows "Click Me" in blue
<Button label="Submit" />       // Shows "Submit" in blue
<Button label="Delete" color="red" />  // Shows "Delete" in red
```

Prop Types (Runtime type-checking)

PropTypes

```
import PropTypes from 'prop-types';

function UserCard({ name, age, isAdmin }) {
  return <div>{name}</div>;
}

UserCard.propTypes = {
  name:    PropTypes.string.isRequired,
  age:     PropTypes.number,
  isAdmin: PropTypes.bool,
};

UserCard.defaultProps = {
  age: 18,
  isAdmin: false,
};
```

The children Prop

children Prop

```
// Card component that wraps any content
function Card({ children, title }) {
  return (
    <div className="card">
      <h3>{title}</h3>
      <div>{children}</div>    {/* Renders whatever is nested inside */}
    </div>
  );
}

// Usage
<Card title="My Card">
  <p>This is the card body!</p>
  <button>Action</button>
</Card>
```

■ 6. State & useState Hook

State represents **dynamic data** that can change over time. When state changes, React **re-renders** the component to reflect the new data. State is **local** to the component that owns it.

■■ **Note:** Never modify state directly (e.g., state.count = 5). Always use the setter function.

useState Hook — Syntax

useState Syntax

```
import { useState } from 'react';

// const [stateVariable, setterFunction] = useState(initialValue);
const [count,   setCount]   = useState(0);
const [name,    setName]    = useState("Alice");
const [isOpen,  setIsOpen]  = useState(false);
const [items,   setItems]   = useState([]);
```

Counter Example

Counter with useState

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);   // Initial value = 0

  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>■ Increment</button>
      <button onClick={() => setCount(count - 1)}>■ Decrement</button>
      <button onClick={() => setCount(0)}>■ Reset</button>
    </div>
  );
}
```

Updating Objects in State

Object State

```
function Profile() {
  const [user, setUser] = useState({ name: "Alice", age: 25 });

  const birthday = () => {
    // ■ Spread old state, then override the field
    setUser({ ...user, age: user.age + 1 });

    // ■ WRONG — mutating state directly
    // user.age = user.age + 1;  // Does not trigger re-render!
  };

  return (
    <div>
      <p>{user.name} is {user.age} years old</p>
```

```
      <button onClick={birthday}>■ Happy Birthday!</button>
    </div>
  );
}
```

```
}
```

■■ 7. Event Handling

React wraps native browser events in **SyntheticEvents** — a cross-browser wrapper that ensures consistent behaviour in all browsers.

React Event	Fires When	HTML Equivalent
onClick	Element is clicked	onclick
onChange	Input value changes	onchange
onSubmit	Form is submitted	onsubmit
onMouseOver	Mouse enters element	onmouseover
onKeyDown	Key is pressed	onkeydown
onFocus	Element gains focus	onfocus
onBlur	Element loses focus	onblur

Basic Event Handling

Event Handling

```
function AlertButton() {
  // ■ Pass a function reference — don't call it with ()
  const handleClick = () => {
    alert("Button clicked!");
  };

  return <button onClick={handleClick}>Click Me</button>;
}

// ■ Inline arrow function
function AlertButton2() {
  return (
    <button onClick={() => alert("Clicked!")}>Click Me</button>
  );
}
```

The Event Object (e)

Event Object

```
function InputLogger() {
  const handleChange = (e) => {
    console.log("Value:", e.target.value);
    console.log("Name:", e.target.name);
    console.log("Type:", e.target.type);
  };

  return (
    <input
      type="text"
      name="username"
      onChange={handleChange}
```

```
      placeholder="Type something..."
    />
  );
}
```

■ 8. Lists & Keys

To render a list of items in React, use JavaScript's **Array.map()** method to transform an array into an array of JSX elements.

Rendering a List

Basic List

```
const fruits = ["Apple", "Banana", "Cherry", "Mango"];

function FruitList() {
  return (
    <ul>
      {fruits.map((fruit, index) => (
        <li key={index}>{fruit}</li>
      ))}
    </ul>
  );
}
```

Using Unique IDs as Keys (Recommended)

List with Unique Keys

```
const users = [
  { id: 101, name: "Alice", role: "Admin" },
  { id: 102, name: "Bob",   role: "Editor" },
  { id: 103, name: "Carol", role: "Viewer" },
];

function UserList() {
  return (
    <table>
      <thead>
        <tr><th>Name</th><th>Role</th></tr>
      </thead>
      <tbody>
        {users.map((user) => (
          <tr key={user.id}>     {/* Always use unique ID as key */}
            <td>{user.name}</td>
            <td>{user.role}</td>
          </tr>
        ))}
      </tbody>
    </table>
  );
}
```

■■ **Note:** Avoid using array index as key if the list can be reordered or items can be deleted.

Why Keys Matter

Keys help React identify which items have changed, been added, or removed during re-renders. A stable, unique key enables efficient DOM updates and preserves component state correctly.

■ 9. Putting It All Together — Mini Counter App

This example combines **components, props, state, and event handling** into a small but complete application.

Counter.js

```
// File: src/components/Counter.js
import { useState } from 'react';

// Child component — receives initialCount via props
function Counter({ initialCount = 0, label = "Counter" }) {
  const [count, setCount] = useState(initialCount);

  const increment = () => setCount(prev => prev + 1);
  const decrement = () => setCount(prev => prev - 1);
  const reset     = () => setCount(initialCount);

  return (
    <div style={{ border: "1px solid #ccc", padding: "16px", margin: "8px" }}>
      <h3>{label}</h3>
      <p style={{ fontSize: "2rem", fontWeight: "bold" }}>{count}</p>
      <button onClick={decrement}>−</button>
      <button onClick={reset} style={{ margin: "0 8px" }}>Reset</button>
      <button onClick={increment}>+</button>
    </div>
  );
}

export default Counter;
```

App.js

```
// File: src/App.js
import Counter from './components/Counter';

function App() {
  return (
    <div>
      <h1>React Counter Demo</h1>
      <Counter label="Score"       initialCount={0} />
      <Counter label="Lives Left"  initialCount={3} />
      <Counter label="High Score"  initialCount={100} />
    </div>
  );
}

export default App;
```

What this demonstrates

- **Component reuse** — Counter is used three times with different props
- **Props** — label and initialCount customize each instance
- **State** — each Counter has its own independent count
- **Event handling** — buttons trigger state updates
- **Functional updater** — setCount(prev => prev + 1) is safer with async updates

■ 10. React Day 1 — Quick Reference Cheat Sheet

Core Concepts

Concept	Syntax / Usage	Purpose
Functional Component	function MyComp() { return ; }	Reusable UI block
JSX Expression	{variable} or {expression}	Embed JS in HTML
Fragment	<>...	Multiple roots without div
className	className='myClass'	CSS classes in JSX
Props (receive)	function C({ name }) { ... }	Data from parent
useState	const [val, setVal] = useState(0)	Local mutable state
Update state	setVal(newValue)	Triggers re-render
Prev state pattern	setVal(prev => prev + 1)	Safe async update
onClick	onClick={handler}	Click event
onChange	onChange={(e) => setVal(e.target.value)}	Input change
List rendering	arr.map(item =>)	Dynamic lists
Conditional (&&)	{condition && }	Show/hide element
Conditional (ternary)	condition ? :	Toggle elements
Default export	export default MyComp	One per file
Named export	export function MyComp() {}	Multiple per file

Common Mistakes to Avoid

- ■ Returning multiple root elements without a wrapper or Fragment
- ■ Using class instead of className for CSS
- ■ Calling a function in onClick: onClick={handleClick()} — remove the ()
- ■ Mutating state directly: state.value = newVal — use the setter
- ■ Forgetting the key prop when rendering lists
- ■ Starting component names with lowercase letters

Next Steps (Day 2 Preview)

- useEffect Hook — side effects, data fetching, timers
- React Router — multi-page navigation
- Lifting State Up — sharing state between sibling components
- Context API — avoiding prop drilling
- Custom Hooks — reusing stateful logic